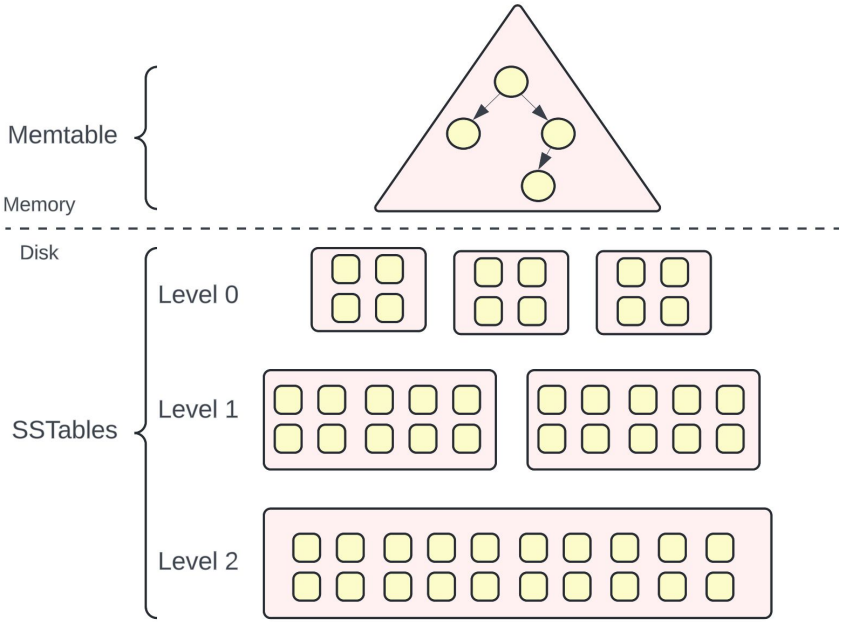
From WiscKey to Bourbon: A Learned Index for Log-Structured Merge Trees

Yifan Dai, Yien Xu, Aishwarya Ganesan, Ramnatthan Alagappan, Brian Kroth
Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau

Presented by: Kaushik Kurapati

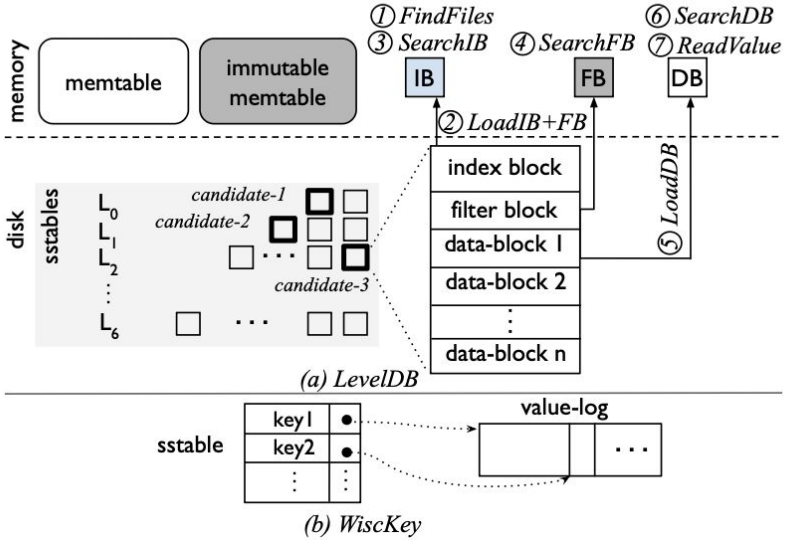
Background: What are LSM Trees

- High-write throughput
- LevelDB, RocksDB, Cassandra
- Data organized:
  - Memtable
  - Multiple disk levels (L0–Ln)
- Writes are sequential
- Reads search from top to bottom



Lookup in LSM

1. Find candidate files
2. Load index block
3. Search index
4. Query Bloom Filter
5. Load data block
6. Binary search inside data block



Insight:

- Lookups involve many internal searches
- Indexing cost becomes significant in memory-resident workloads

LSM and Learned Indexes

Problem:

- Learned indexes assume static datasets
- LSMs are optimized for writes
- Writes change data distribution
- Would require constant retraining of learned indexes

So this brings up the questions:

1. Can learned indexes for LSMs make lookups faster? If yes, under what scenarios?
2. How to realize the benefits of learned indexes while supporting writes for which LSMs are optimized?

Key Observation

- Most data resides in lower levels
- SSTables are immutable
- Immutable and sorted make it perfect for learning

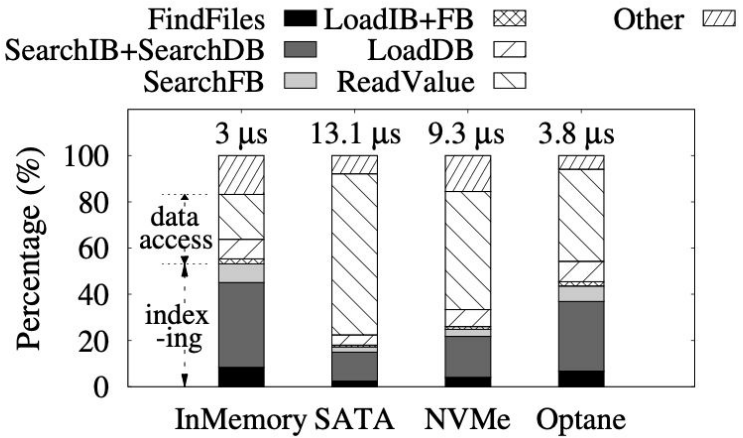
Learned Indexes: Beneficial Regimes

Learning helps when:

- Data is cached in memory
- Storage is fast (Optane, NVMe)
- Indexing dominates latency

Gains:

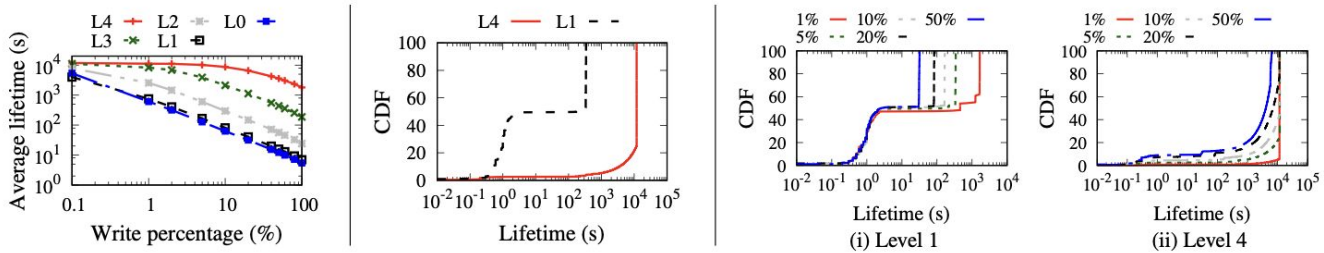
- Indexing cost up to 44% of lookup time, learned indexes can have a potential 1.8x improvement
- For caching, 2x performance gain



Learned Indexes with Writes

They measure:

- SSTable lifetimes
- Level lifetimes
- Internal lookup distribution
- Negative vs positive lookups
- Effect of write ratio



(a) Average lifetimes with varying write % (b) Lifetime distribution with 5% writes

(c) Lifetime distributions with varying write %

Five learning guidelines

1. Favor learning files at lower levels
2. Wait before learning a file
3. Don’t neglect files at higher levels
4. Be workload- and data-aware
5. Do not learn levels for write-heavy workloads

Bourbon Design

What is Bourbon?

- Learned-index LSM built on WiscKey
- Uses file-level learning
- Uses piecewise linear regression (PLR)

Key design choice:

- Keys fixed-size
- Values stored separately (WiscKey style)

Piecewise Linear Regression

Why PLR?

- Linear-time training
- Small space overhead
- Simple inference
- Predictable error bounds

Which explains:

- Model predicts position
- Error bound δ
- Local search within $[pos - \delta, pos + \delta]$

File vs Level Learning

File Learning	Level Learning
More stable	Faster interference
Works with writes	Fails under heavy writes
Better for most workloads	Good for read-only

Workload	Baseline time (s)	File model		Level model	
		Time(s)	% model	Time(s)	% model
Mixed: Write-heavy	82.6	71.5 (1.16 ×)	74.2	95.1 (0.87 ×)	1.5
Mixed: Read-heavy	89.2	62.05 (1.44 ×)	99.8	74.3 (1.2 ×)	21.4
Read-only	48.4	27.2 (1.78 ×)	100	25.2 (1.92 ×)	100

Bourbon uses file learning, but supports level learning (enabled in config options)

Cost-Benefit Analyzer

Learn file if:

B_model > C_model

Where:

C_model: Cost to build the model (C_model = T_build)

- T_build is the time to train the PLR model for a file.
- linearly proportional to the number of datapoints in the file.
- Calculate by multiplying the average time to a train a data point (measured offline) and the number of data points in the file.

B_model: Benefit of the model

B_model

$$B_{model} = ((T_{n.b} - T_{n.m}) * N_n) + ((T_{p.b} - T_{p.m}) * N_p)$$

$T_{n.b}$: Time spent on **negative lookups** in the **baseline** (no model).

$T_{n.m}$: Time spent on **negative lookups** using the **model**.

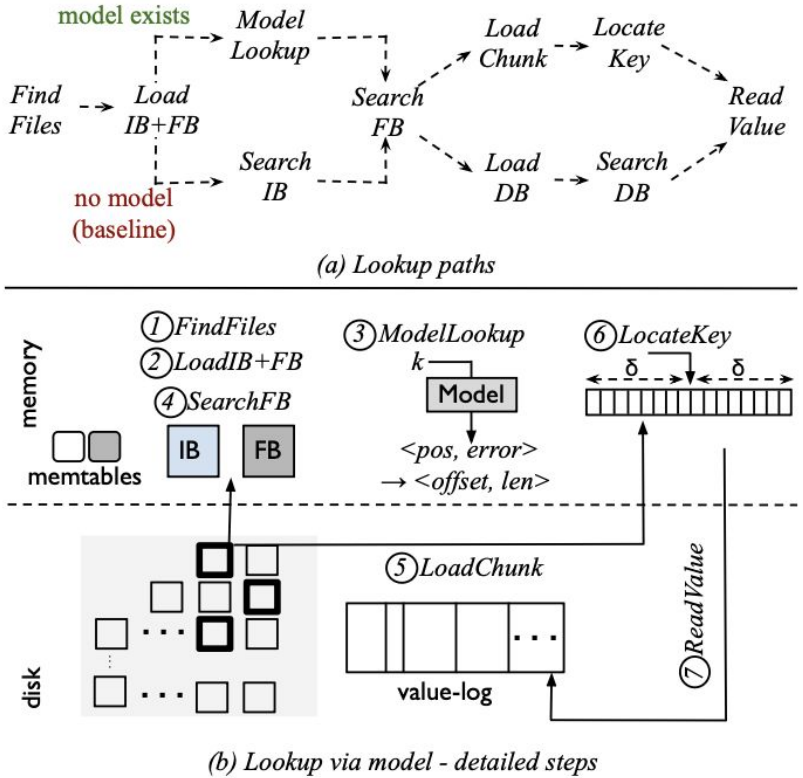
$T_{p.b}$: Time spent on **positive lookups** in the **baseline**.

$T_{p.m}$: Time spent on **positive lookups** using the **model**.

N_n: Number of **negative lookups** the file serves during its lifetime.

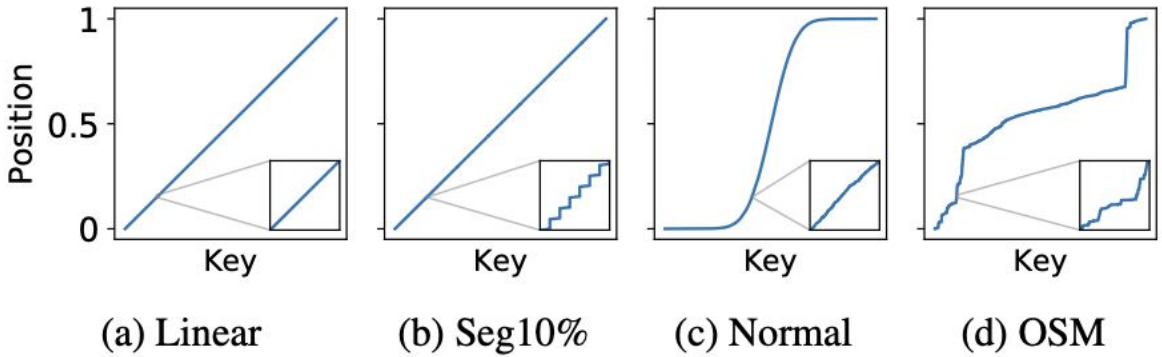
N_p: Number of **positive lookups** the file serves during its lifetime.

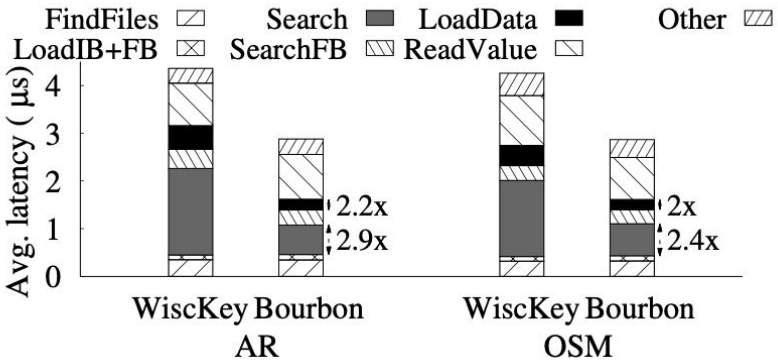
BOURBON

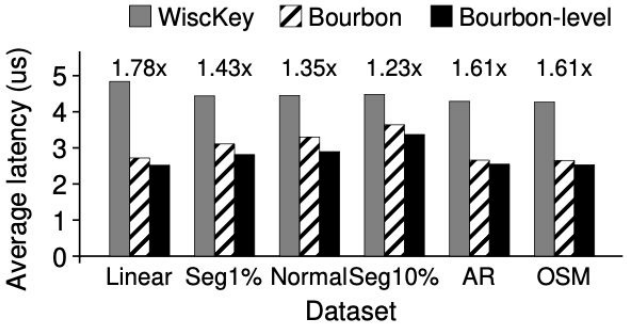

model exists
Model Lookup
Load Chunk
Locate Key
Find Files
Load IB+FB
Search FB
Read Value
Search IB
Load DB
Search DB
no model (baseline)
(a) Lookup paths
memory
1 FindFiles
2 LoadIB+FB
4 SearchFB
3 ModelLookup
k
Model
6 LocateKey
δ
δ
IB
FB
memtables
<pos, error>
→ <offset, len>
5 LoadChunk
7 ReadValue
disk
value-log
(b) Lookup via model - detailed steps

Evaluation

- 4 synthetic datasets
  - linear, segmented-1%, segmented-10% , and normal, each with 64M key-value pairs.
- 2 real-world datasets
  - Amazon reviews (AR) and New York OpenStreetMaps







(a) Average lookup latency

Dataset	#segs	latency (μs)
Linear	900	2.72
Seg1%	640K	3.11
Normal	705K	3.3
Seg10%	6.4M	3.64
AR	129K	2.66
OSM	295K	2.65

(b) Number of segments

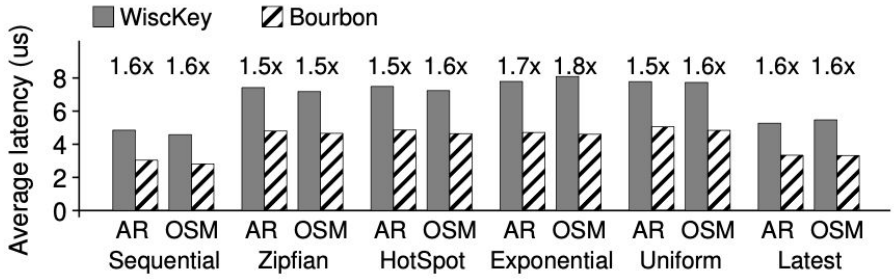


Figure 11: **Request Distributions.** *The figure shows the average lookup latencies of different request distributions from AR and OSM datasets.*

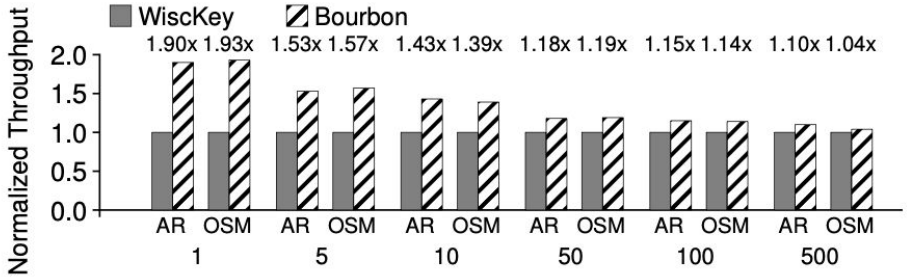
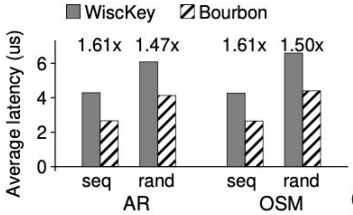


Figure 12: **Range Queries.** *The figure shows the normalized throughput of range queries with different range lengths from AR and OSM datasets.*



(a) Average latency

Dataset	Positive		Negative	
	#	Speedup	#	Speedup
AR	10M	2.15×	23M	1.83×
OSM	10M	1.99×	22M	1.82×

(b) Positive vs. negative internal lookups for randomly loaded case

Figure 10: **Load Orders.** *(a) shows the performance for AR and OSM datasets for sequential (seq) and random (rand) load orders. (b) compares the speedup of positive and negative internal lookups.*

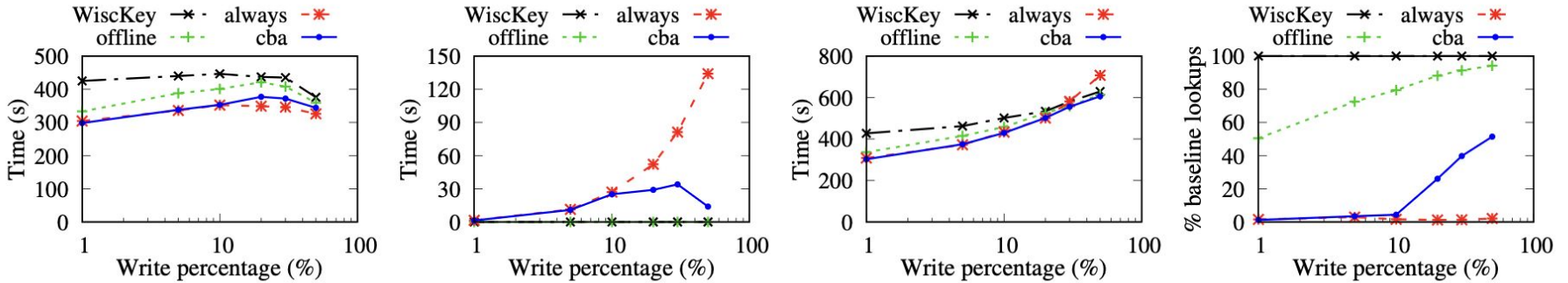


Figure 13: **Mixed Workloads.** *(a) compares the foreground times of WiscKey,* BOURBON*-offline (offline),* BOURBON*-always (always), and* BOURBON*-cba (cba); (b) and (c) compare the learning time and total time, respectively; (d) shows the fraction of internal lookups that take the baseline path.*

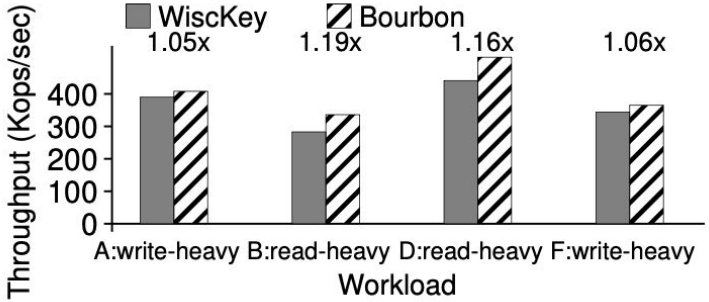


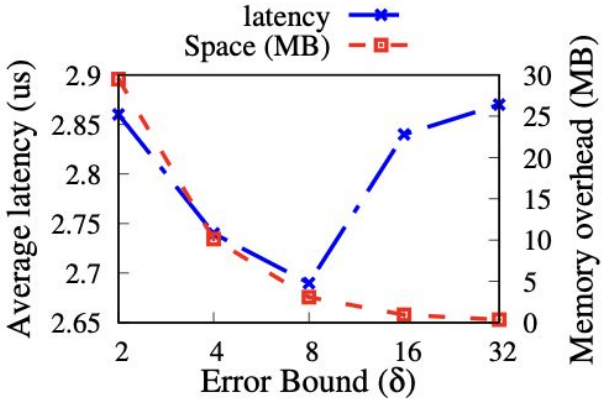
Figure 16: **Mixed Workloads on Fast Storage.** *The figure compares the throughput of* BOURBON *against WiscKey for four read-write mixed YCSB workloads. We use the YCSB default dataset for this experiment.*

Dataset	WiscKey latency (μs)	BOURBON	
		Latency(μs)	Speedup
Amazon Reviews (AR)	3.53	2.75	1.28×
NewYork OpenStreetMaps (OSM)	3.14	2.51	1.25×

Table 2: **Performance on Fast Storage.** *The table shows* BOURBON*'s lookup latencies when the data is stored on an Optane SSD.*

Workload	WiscKey latency (μs)	BOURBON	
		Latency(μs)	Speedup
Uniform	98.6	94.4	1.04×
Zipfian	18.8	15.1	1.25×

Table 3: **Performance with Limited Memory.** *The table shows* BOURBON*'s average lookup latencies from the AR dataset on a machine with a SATA SSD and limited memory.*



(a) Error-bound tradeoff

Dataset	Space Overheads	
	MB	%
Linear	0.02	0.0
Seg1%	15.38	0.21
Seg10%	153.6	2.05
Normal	16.94	0.23
AR	3.09	0.08
OSM	7.08	0.26

(b) Space overheads

Figure 17: **Error-bound Tradeoffs and Space Overheads.** *(a) shows how the PLR error bound affects lookup latency and memory overheads; (b) shows the space overheads for different datasets.*

Strengths & Weaknesses

Strengths:

- Empirical analysis
- Clear system guidelines
- Practical implementation
- Real system integration
- Cost-benefit modeling
- Works under writes

Weaknesses:

- Only integer fixed-size keys
- No adaptive model switching
- Only PLR evaluated
- Assumes statistics from previous files are predictive

Future Work & Implications

Future Work:

- Support variable-length string keys
- Smarter cost models (workload prediction)
- Hybrid model types

Implications:

- Learned systems must be workload-aware
- Clear guidelines
- Application of ML in a beneficial way